

1 Explainability in Reinforcement Learning

2 Maxime Alaarabiou

3 Thursday 10th September, 2026

Chapter 1

Experiments

This chapter reports the experimental study of the Clustering Agent Representation (CAR) method introduced in Chapter ???. While the previous chapter presented CAR and its evaluation metrics from a theoretical standpoint, this chapter focuses on their concrete realization: the software used to run the experiments, the environments and training setups considered, and a detailed analysis of the results obtained.

This chapter is organized as follows:

- Section 1.1 describes the software infrastructure implementing the CAR pipeline.
- Section ?? details the experimental configuration, including the environments, the reinforcement learning setup, and the training of the surrogate policies.
- Section ?? presents and analyses the results obtained by applying CAR and its metrics.
- Section ?? summarizes the findings and discusses their implications.

1.1 Software Infrastructure

Deep Learning (DL) and Reinforcement Learning (RL) are inherently empirical fields, in which the effectiveness of a method is ultimately assessed through experimentation. Experimental validation is therefore as important as the theoretical formulation of the method. For this reason, particular attention is given to the implementation of CAR. The method relies on established libraries that provide efficient and widely used implementations of its main components. The implementation is made publicly available as open-source software to support the objective and reproducible replication of the reported results.¹

CAR is organized as a pipeline of five stages, shown in Figure 1.1. Each stage is an independent executable, and stages communicate only through artifacts written to disk. This decoupling allows any stage to be rerun or inspected independently. The parameters of each experiment are gathered in a

¹https://github.com/Maxalaar/Clustering_Agent_Latent_Space

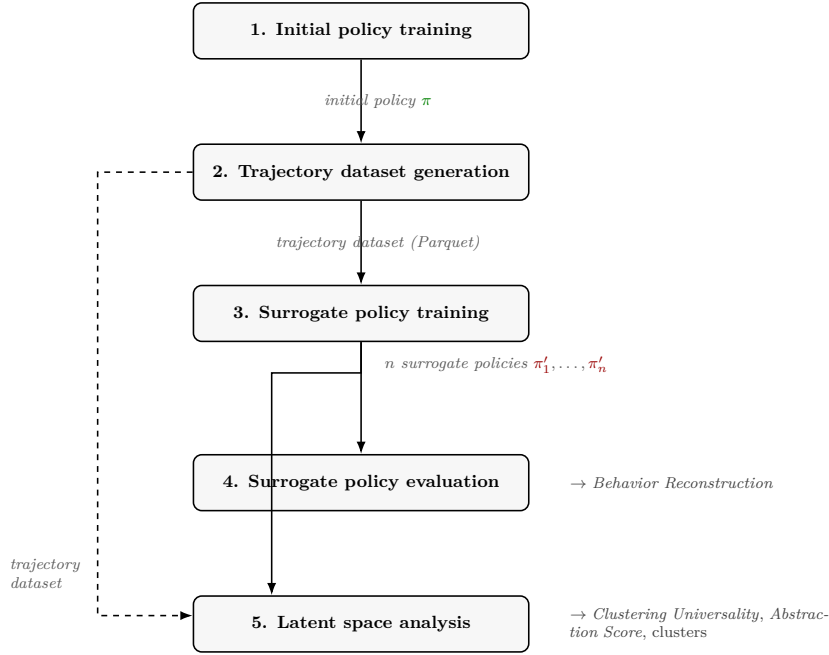


Figure 1.1: The CAR pipeline. Each stage is an independent executable; stages communicate only through artifacts written to disk (solid arrows). The dashed arrow indicates an artifact reused by a later stage.

34 single configuration object, described in Section ?? . Each stage also saves the
 35 configuration it used alongside its outputs. Every result can therefore be traced
 36 back to the settings that produced it.

37 1.1.1 Initial policy training

38 The first stage trains the initial policy π using RLlib [Moritz et al., 2018, Liang et al., 2018].
 39 RLlib provides a highly scalable implementation of RL algorithms and supports a
 40 wide range of state-of-the-art methods. It supports distributed training, allowing
 41 the computational workload to be spread across multiple processes and machines.

42 The use of RLlib allows CAR to be applied to any environment implemented
 43 using the Gymnasium interface [Towers et al., 2024]. This provides a common
 44 interface for environments with different observation and action spaces. This
 45 flexibility allows CAR to be applied to a wide range of environments.

46 By default, the RL algorithm used for the initial policy is Proximal Policy Op-
 47 timization (PPO) [Schulman et al., 2017]. Other algorithms supported by RLlib,
 48 such as Asynchronous Proximal Policy Optimization (APPO) [Espeholt et al., 2018],
 49 Deep Q-Networks (DQN) [Mnih et al., 2013], and DreamerV3, can also be se-
 50 lected through the same configuration.

51 The policy and critic networks are implemented using PyTorch [Paszke et al., 2019].
 52 PyTorch provides efficient GPU computation and supports a wide range of neural
 53 network architectures. This provides a standardized implementation that is
 54 compatible with the broader DL ecosystem.

55 The output of this stage is a checkpoint containing the trained policy π .

1.1.2 Trajectory dataset generation

The second stage builds the dataset used to train the surrogate policies. The distributed execution capabilities of RLlib are used to collect trajectories in parallel. Multiple environment runners execute the original policy simultaneously, distributing the computational workload across the available resources.

The original policy is run in the environment, and the interactions between the agent and the environment are recorded. For each step, the dataset stores the observation together with the parameters of the action distribution produced by the policy. Trajectories are stored as Parquet files [The Apache Software Foundation, 2016]. Parquet is a compressed columnar format widely used for large-scale data processing. Its columnar structure allows data to be read efficiently and supports concurrent access from multiple processes. This is particularly useful here because the datasets contain on the order of one million observation–action pairs and cannot be loaded entirely into memory.

1.1.3 Surrogate policy training

The third stage trains the surrogate policies using behavior cloning. This stage therefore converts the original RL problem into a supervised learning problem. The surrogate policies are trained to reproduce the action distributions generated by π from the collected observations.

The training procedure is implemented using PyTorch Lightning [Falcon and The PyTorch Lightning team, 2019]. PyTorch Lightning provides a structured interface for supervised training while remaining compatible with the PyTorch ecosystem. It handles common training operations such as data loading, optimization, checkpointing, and GPU execution, while allowing the underlying neural network implementation to remain fully controlled.

Once training is complete, the stage produces a set of n trained surrogate policies.

1.1.4 Surrogate policy evaluation

The fourth stage measures how faithfully each surrogate policy reproduces the behavior of π . Each surrogate is wrapped so that it can be used as an RLlib policy and is then evaluated in the original environment. Its mean cumulative reward over multiple episodes is compared with that of π . Evaluation episodes are collected using parallel environment runners.

This stage provides the value of the *Behavior Reconstruction*. It therefore verifies whether the surrogate policies adequately reproduce the behavior of the original policy before their latent representations are analyzed. If a surrogate does not reproduce the original behavior sufficiently well, its training can be repeated before proceeding to the next stage.

1.1.5 Latent space analysis

The fifth stage performs the main analysis of CAR. The latent representations produced by the surrogate policies form the point clouds that are analyzed using clustering algorithms.

The clustering step relies on the cuML library [RAPIDS Development Team, 2023], which provides GPU-accelerated implementations of several clustering algorithms.

100 This significantly reduces the computational cost of clustering and makes it pos-
101 sible to analyze point clouds containing tens of thousands of representations.

102 The resulting partitions are used to compute the *Clustering Universality* and
103 the *Abstraction Score*. The outputs of this stage are the resulting metric values
104 and the material required for the qualitative analysis presented in the following
105 sections.

106 Acronyms

107 **APPO** Asynchronous Proximal Policy Optimization 2

108 **CAR** Clustering Agent Representation 1–3

109 **DL** Deep Learning 1, 2

110 **DQN** Deep Q-Networks 2

111 **PPO** Proximal Policy Optimization 2

112 **RL** Reinforcement Learning 1–3

Bibliography

- [Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Falcon and The PyTorch Lightning team, 2019] Falcon, W. and The PyTorch Lightning team (2019). PyTorch Lightning. <https://github.com/Lightning-AI/pytorch-lightning>.
- [Liang et al., 2018] Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J. E., Jordan, M. I., and Stoica, I. (2018). RLlib: Abstractions for distributed reinforcement learning. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.
- [Moritz et al., 2018] Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), pages 561–577.
- [Paszke et al., 2019] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In Advances in Neural Information Processing Systems (NeurIPS).
- [RAPIDS Development Team, 2023] RAPIDS Development Team (2023). RAPIDS cuML: GPU-accelerated machine learning. <https://github.com/rapidsai/cuml>.
- [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms.
- [The Apache Software Foundation, 2016] The Apache Software Foundation (2016). Apache Arrow and Parquet. <https://arrow.apache.org>.

- 147 [Towers et al., 2024] Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U.,
148 De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG,
149 A., et al. (2024). Gymnasium: A standard interface for reinforcement learning
150 environments.